

Documento de Arquitetura de Software

Extinction

Gestor do Projeto	Gerente de Projeto
Daniel Souza de Lima	[nome]
[danielsouzalimabsb@gmail.com]	[email]
[6198108-0783]	[telefone]

Objetivo deste Documento

Este documento tem como objetivo descrever as principais decisões de projeto tomadas pela equipe de desenvolvimento e os critérios considerados durante a tomada destas decisões. Suas informações incluem a parte de *hardware* e *software* do sistema.

Histórico de Revisão

Data	Demanda	Autor	Descrição	Versão
[17/05/2026]	[XX000000]	[Daniel Souza de Lima]	[Casos de Uso]	[1.0.1]
[28/05/2026]		[Guilherme Alves]		
		[Cayky]		

SUMÁRIO

1. INTRODUÇÃO	3
1.1 Finalidade	3
1.2 Escopo	3
1.3 Definições, Acrônimos e Abreviações	3
1.4 Referências	4
2. REPRESENTAÇÃO ARQUITETURAL	4
3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS	5
4. VISÃO DE HISTÓRIAS DE USUÁRIO	6
5. VISÃO LÓGICA	8
5.1 Visão Geral – pacotes e camadas	8
6. VISÃO DE IMPLEMENTAÇÃO	11
6.1 Casos de Uso	11
6.1.1 <i>Diagrama de Classes</i>	14
6.1.2 <i>Diagrama de Sequência</i>	15
7. VISÃO DE IMPLANTAÇÃO	20
8. DIMENSIONAMENTO E PERFORMANCE	20
8.1 Volume	20
8.2 Performance	21
9. QUALIDADE	22

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema Extinction, usando diversas visões de arquitetura para **representar** diferentes aspectos do sistema. O objetivo deste documento é capturar e comunicar as decisões arquiteturais significativas que foram tomadas em relação ao sistema.

O documento irá adotar uma estrutura baseada na visão “4+1” de modelo de arquitetura [KRU41].



Figura 1 – Arquitetura 4+1

1.2 Escopo

Este Documento de Arquitetura de Software se aplica ao Extinction, que será desenvolvido pela equipe Daniel Souza, Flávio Daniel, Guilherme Alves, Alysson Kauã, Cayky, André Ribamar.

1.3 Definições, Acrônimos e Abreviações

QoS – Quality of Service, ou qualidade de serviço. Termo utilizado para descrever um conjunto de qualidades que descrevem os requisitos não-funcionais de um sistema, como performance, disponibilidade e escalabilidade[QOS].

1.4 Referências

- [KRU41]: The “4+1” view model of software architecture, Philippe Kruchten, November 1995, <http://www3.software.ibm.com/ibmdl/pub/software/rational/web/whitepapers/2003/Pbk4p1.pdf>
- [QOS] <https://docs.oracle.com/cd/E19636-01/819-2326/6n4kfe7dj/index.html>

2. REPRESENTAÇÃO ARQUITETURAL

Este documento irá detalhar as visões baseado no modelo “4+1” [KRU41], utilizando como referência os modelos definidos na MDS. As visões utilizadas no documento serão:

Visão	Público	Área	Modelo da MDS
Lógica	Analistas	Realização dos Casos de Uso	Casos de Uso, Diagrama de Camadas da Aplicação, Diagrama de Pacotes da Aplicação, Diagrama de Classes, Diagrama de Sequência.
Processo	Integradores	Performance, Escalabilidade, Concorrência	Diagrama de Camadas.
Implementação	Programadores	Componentes de Software	Diagrama de Pacotes, Estrutura de Diretórios/Repositório.
Implantação	Gerência de Configuração	Nodos físicos	Diagrama de Implantação, Arquitetura de Infraestrutura.
Caso de Uso	Todos	Requisitos funcionais	Diagrama de Casos de Uso (Histórias de Usuário).
Dados	Especialistas em dados Administradores de dados	Persistência de dados	Modelo Entidade-Relacionamento (MER), Diagrama.

Tabela 1 – Visões, Público, Área e Artefatos da MDS

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descrever os requisitos de software e restrições que tem um impacto significativo na arquitetura.

Requisito	Solução
Linguagem	Java (Versão 17 LTS ou superior), garantindo estabilidade, tipagem forte e suporte a recursos modernos de programação funcional e concorrência para o processamento de regras de negócio.
Plataforma	Spring Boot (Versão 3.4.1), utilizando seu ecossistema modular para acelerar a configuração, simplificar a injeção de dependências e expor endpoints RESTful eficientes através do ecossistema <i>Spring Web</i> .
Segurança	Spring Security + JWT (JSON Web Tokens): <i>Necessidade:</i> Autenticar usuários de forma segura, proteger dados de perfil e restringir endpoints confidenciais (como cadastro de espécies em extinção ou logs de auditoria) com base no nível de acesso do usuário (COMUM, CIENTISTA, ADMIN). <i>Características Básicas:</i> Implementação de autenticação <i>stateless</i> (sem estado no servidor). Após o login com sucesso, a API emite um token JWT assinado digitalmente com algoritmo robusto (HMAC256 ou RSA256). Cada requisição subsequente envia o token no cabeçalho Authorization: Bearer. O controle de acessos dentro do código é feito de forma declarativa através de anotações como <code>@PreAuthorize("hasRole('ADMIN')")</code> nos controladores REST. Senhas são criptografadas antes de salvar no banco utilizando o algoritmo de hash adaptável BCryptPasswordEncoder.
Persistência	Spring Data JPA + Hibernate: <i>Necessidade:</i> Garantir a persistência de dados de forma transacional (propriedades ACID), gerenciando os registros de usuários, mídias de postagens e metadados biológicos com integridade referencial e alta performance de leitura para o mapa de avistamentos. <i>Mecanismo Adotado:</i> Utilização do ecossistema <i>Spring Data JPA</i> com o <i>Hibernate</i> como provedor do Mecanismo de Mapeamento Objeto-Relacional (ORM). Toda a manipulação de dados ocorre através do padrão <i>Repository</i> , isolando comandos SQL manuais. Para otimização de performance, o pool de conexões padrão HikariCP é configurado no arquivo application.yml para gerenciar a reutilização de conexões abertas com o banco de dados MySQL. O controle transacional é feito usando a anotação <code>@Transactional</code> nas classes de serviço.
Internacionalização (i18n)	Spring MessageSource + AcceptHeaderLocaleResolver: <i>Necessidade:</i> Como a plataforma lida com biodiversidade e espécies ameaçadas, há potencial de uso por pesquisadores internacionais. O sistema precisa traduzir de forma dinâmica as mensagens de erro de validação (ex: "Idade mínima inválida"), respostas das APIs e notificações por e-mail. <i>Mecanismo Adotado:</i> Criação de arquivos de propriedades centralizados na pasta de recursos da aplicação (messages.properties para o padrão/inglês,

	messages_pt_BR.properties para o português brasileiro). O Spring Boot intercepta automaticamente o cabeçalho HTTP Accept-Language enviado pelo navegador ou app mobile do usuário através do componente AcceptHeaderLocaleResolver, injetando em tempo de execução o idioma correto nas mensagens de validação do <i>Spring Validation</i> (como <code>@NotBlank(message = "{usuario.nome.obrigatorio}")</code>).
--	--

Tabela 2 – Exemplo de requisitos e restrições

4. VISÃO DE HISTÓRIAS DE USUÁRIO

Esta seção lista as especificações centrais e significantes para a arquitetura do sistema.
Lista de histórias de usuário do sistema:

4.1 HU01: Cadastro de Usuário

Cenário 1.1: Registro de Novo Usuário com sucesso

Cenário 1.2: Registro de Novo Usuário / sem preencher o nome

Cenário 1.3: Registro de Novo Usuário / sem preencher o nome corretamente

Cenário 1.4: Registro de Novo Usuário / sem preencher o email

Cenário 1.5: Registro de Novo Usuário / sem preencher o email valido

Cenário 1.6: Registro de Novo Usuário / preencher o email já cadastrado

Cenário 1.7: Registro de Novo Usuário / sem informar a data

Cenário 1.8: Registro de Novo Usuário / informar a data futura

Cenário 1.9: Registro de Novo Usuário / informar a data inferior

Cenário 1.10: Registro de Novo Usuário / sem informar a senha

Cenário 1.11: Registro de Novo Usuário / informar a senha com menos de 8 caracteres

Cenário 1.12: Registro de Novo Usuário / informar a senha sem números

Cenário 1.13: Registro de Novo Usuário / informar a senha sem letras

Cenário 1.14: Registro de Novo Usuário / sem informar o campo confirmar senha

Cenário 1.15: Registro de Novo Usuário / informar o campo confirmar senha com valor diferente

Cenário 1.16: Registro de Novo Usuário / validação de campos obrigatórios

Cenário 1.17: Registro de Novo Usuário / Google

Cenário 1.18: Registro de Novo Usuário / Apple

Cenário 1.19: Registro de Novo Usuário / Botão Entrar

4.2 HU02: Login de Usuário

Cenário 2.1: Login / com sucesso

Cenário 2.2: Login / sem preencher o e-mail

Cenário 2.3: Login / preencher o e-mail inválido

Cenário 2.4: Login / sem preencher a senha

Cenário 2.5: Login / preencher a senha incorreta

Cenário 2.6: Login / Google

Cenário 2.7: Login / Apple

- Cenário 2.8: Login / Salvar senha**
- Cenário 2.9: Login / Redirecionar para outra tela (Esqueci minha senha)**
- Cenário 2.10: Login / Redirecionar para outra tela (Não tem uma conta?)**
- Cenário 2.11: Login / Validação de campos obrigatórios**
- Cenário 2.12: Login / Alerta de credenciais incorretas - Tentativas 1 a 5**
- Cenário 2.13: Login / Bloqueio temporário por excesso de tentativas**
- Cenário 2.14: Login / Recusar por usuário não encontrado**
- Cenário 2.15: Login / Recusar login com Google por falha na autenticação**
- Cenário 2.16: Login / Recusar login com Apple por falha na autenticação**

4.3 HU03: Recuperação de Senha

- Cenário 3.1: Recuperação de Senha / com sucesso**
- Cenário 3.2: Recuperação de Senha / sem preencher o email**
- Cenário 3.3: Recuperação de Senha / e-mail inválido**
- Cenário 3.4: Recuperação de Senha / email não cadastrado**
- Cenário 3.5: Recuperação de Senha / validação de campos obrigatórios**
- Cenário 3.6: Recuperação de Senha / voltar para login**

4.4 HU04: Registro de Avistamento com foto

- Cenário 4.1: Seleção de Imagem para Publicação**
- Cenário 4.2: Formato de Arquivo Inválido**
- Cenário 4.3: Publicação sem Título da Espécie**
- Cenário 4.4: Localização Inválida**
- Cenário 4.5: Cancelamento da Publicação**

4.5 HU05: Localização Geográfica de Espécies no Mapa

- Cenário 5.1: Busca no Mapa - OpenStreetMap**
- Cenário 5.2: Busca por Nome Científico**
- Cenário 5.3: Busca sem Resultados Encontrados**
- Cenário 5.4: Agrupamento de Marcadores próximos (Clustering)**
- Cenário 5.5: Carregamento inicial do mapa interativo com sucesso**
- Cenário 5.6: Interação fluida com a navegação do mapa**
- Cenário 5.7: Filtrar mapa por categorias biológicas ou níveis de extinção**
- Cenário 5.8: Refinar visualização geográfica por Estado e Cidade**
- Cenário 5.9: Visualizar prévia da espécie pelo marcador flutuante**
- Cenário 5.10: Abrir detalhes da enciclopédia a partir do cartão do mapa**

4.6 HU06: Acompanhamento de Atualizações no Feed

- Cenário 6.1: Navegação**
- Cenário 6.2: Filtro de Busca**
- Cenário 6.3: Interação: Curtir/Favoritar**
- Cenário 6.4: Visualização imediata de novo avistamento no feed**
- Cenário 6.5: Clique na Notificação Push para Acesso Direto**
- Cenário 6.6: Update Automático em Tempo Real**

4.7 HU07: Consulta Técnico na Enciclopédia Digital

- Cenário 7.1: Visualizar Detalhes**
- Cenário 7.2: Acesso a Fontes Externas de Validação**
- Cenário 7.3: Alerta de Baixa Acurácia (Dados Divergentes)**
- Cenário 7.4: Carregamento da vitrine de imagens da enciclopédia**
- Cenário 7.5: Navegação dinâmica por rolagem infinita no mural**
- Cenário 7.6: Buscar ou filtrar espécies com truncamento de texto longo**
- Cenário 7.7: Tratamento de busca ou filtro sem resultados encontrados**
- Cenário 7.8: Carregamento de detalhes biológicos completos com metadados**
- Cenário 7.9: Ocultar mini-mapa por ausência de dados de geolocalização**
- Cenário 7.10: Tratamento de falha crítica no carregamento de imagens**

4.8 HU08: Alteração de Perfil

- Cenário 8.1: Edição de dados bem-sucedida**
 - Cenário 8.2: Violação de regra de formatação no nome**
 - Cenário 8.3: Usuário atinge o limite de caracteres na Biografia**
 - Cenário 8.4: Usuário clica em Cancelar após edições**
 - Cenário 8.5: Tentativa de upload de foto acima do limite**
 - Cenário 8.6: Upload de arquivo em formato inválido/corrompido**
 - Cenário 8.7: Abertura do fluxo de alterar senha**
 - Cenário 8.8: Erro de senha atual incorreta no Modal de Senha**
 - Cenário 8.9: Troca de senha bem-sucedida**
 - Cenário 8.10: Solicitação de exclusão de conta - Double opt-in**
 - Cenário 8.11: Usuário desiste e cancela a exclusão de conta**
 - Cenário 8.12: Erro crítico de servidor ao carregar tela**
 - Cenário 8.13: Erro de servidor durante upload de foto válida**
 - Cenário 8.14: Erro de banco de dados/conexão ao salvar alterações textuais**
 - Cenário 8.15: Erro interno de servidor ao tentar processar a exclusão**
 - Cenário 8.16: Queda de internet logo após o clique em Logout**
- Casos de Uso significantes para a arquitetura**

5. VISÃO LÓGICA

Descrever uma visão lógica da arquitetura. Descrever as classes mais importantes, sua organização em pacotes de serviços e subsistemas, e a organização desses subsistemas em camadas. Também descreve as realizações dos casos de uso mais importantes, por exemplo, aspectos dinâmicos da arquitetura. Diagramas de classes e sequência devem ser incluídos para ilustrar os relacionamentos entre as classes significativas na arquitetura, subsistemas, pacotes e camadas.

5.1 Visão Geral – pacotes e camadas

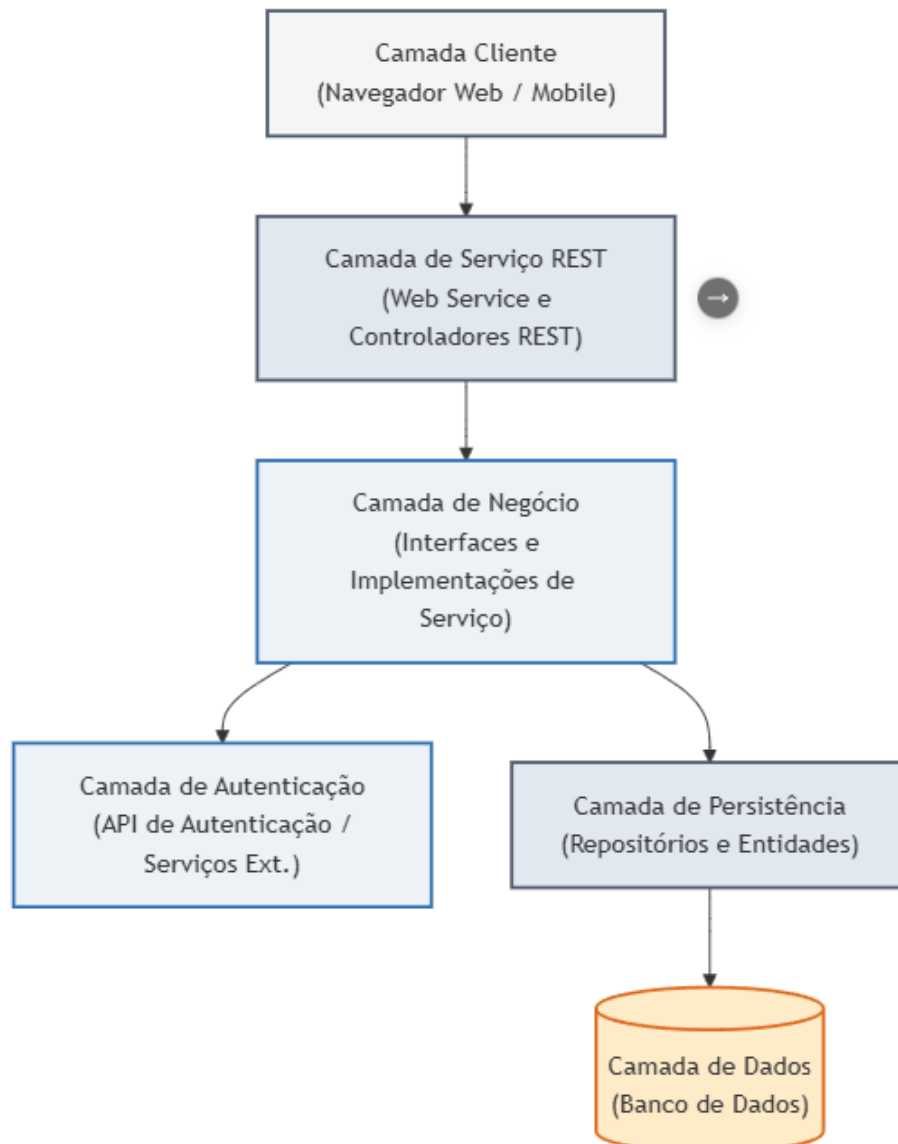


Figura 2 –Diagrama de Camadas da Aplicação

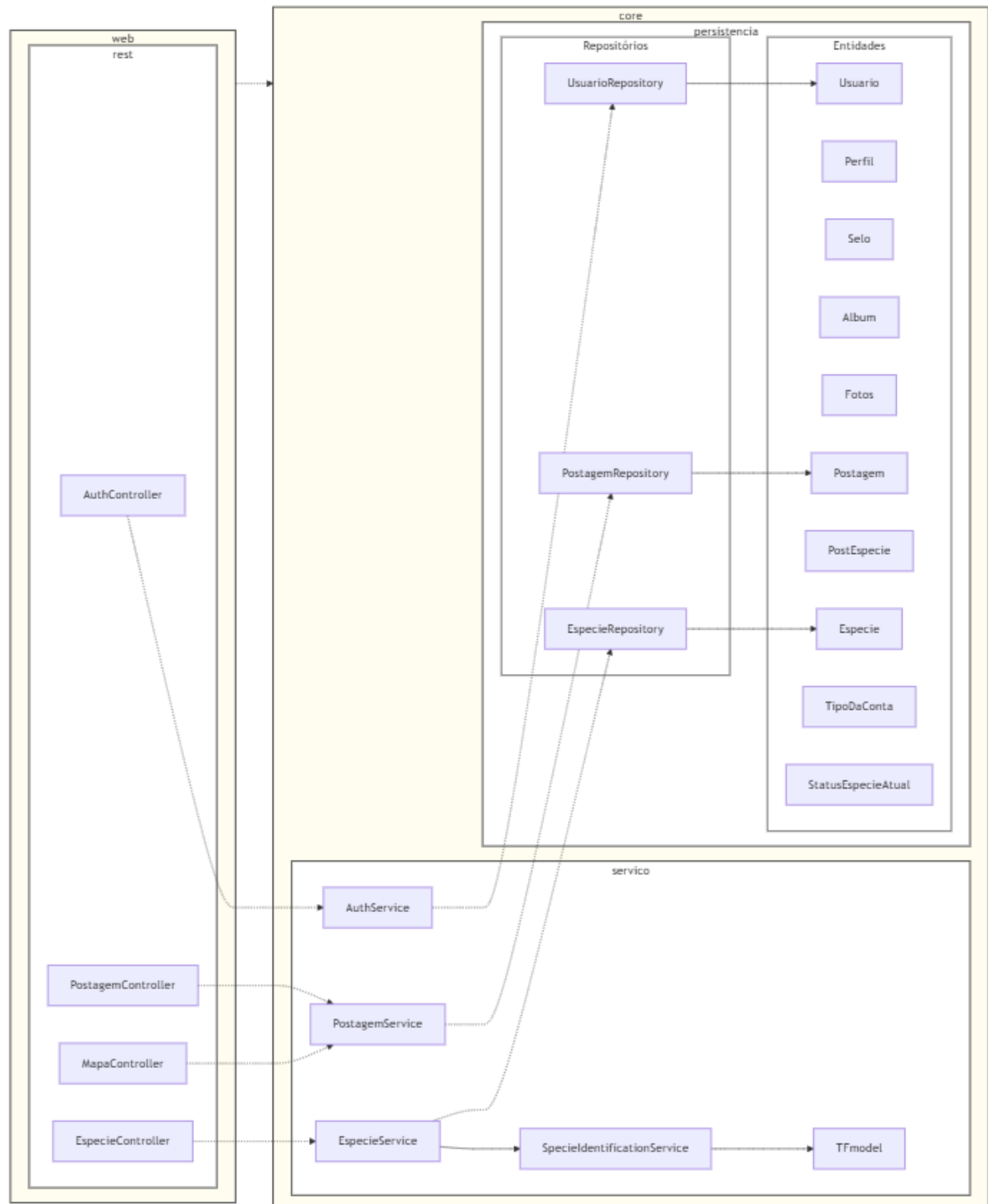
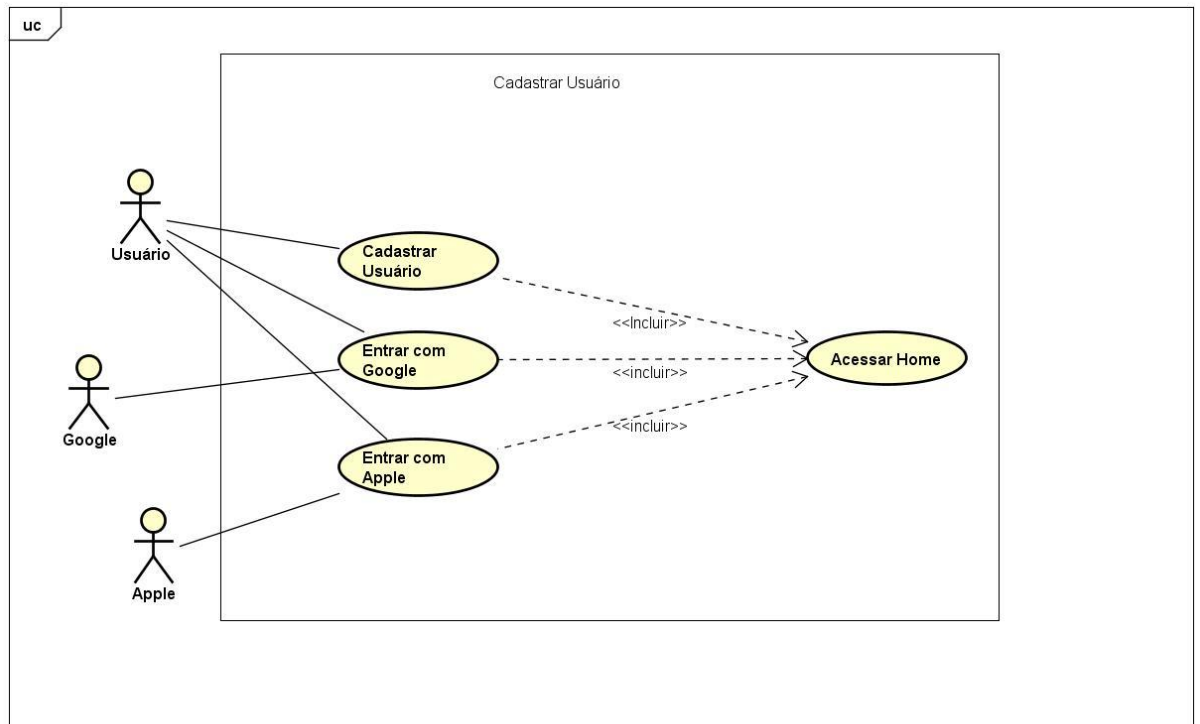


Figura 3 – Diagrama de Pacotes da Aplicação

6. VISÃO DE IMPLEMENTAÇÃO

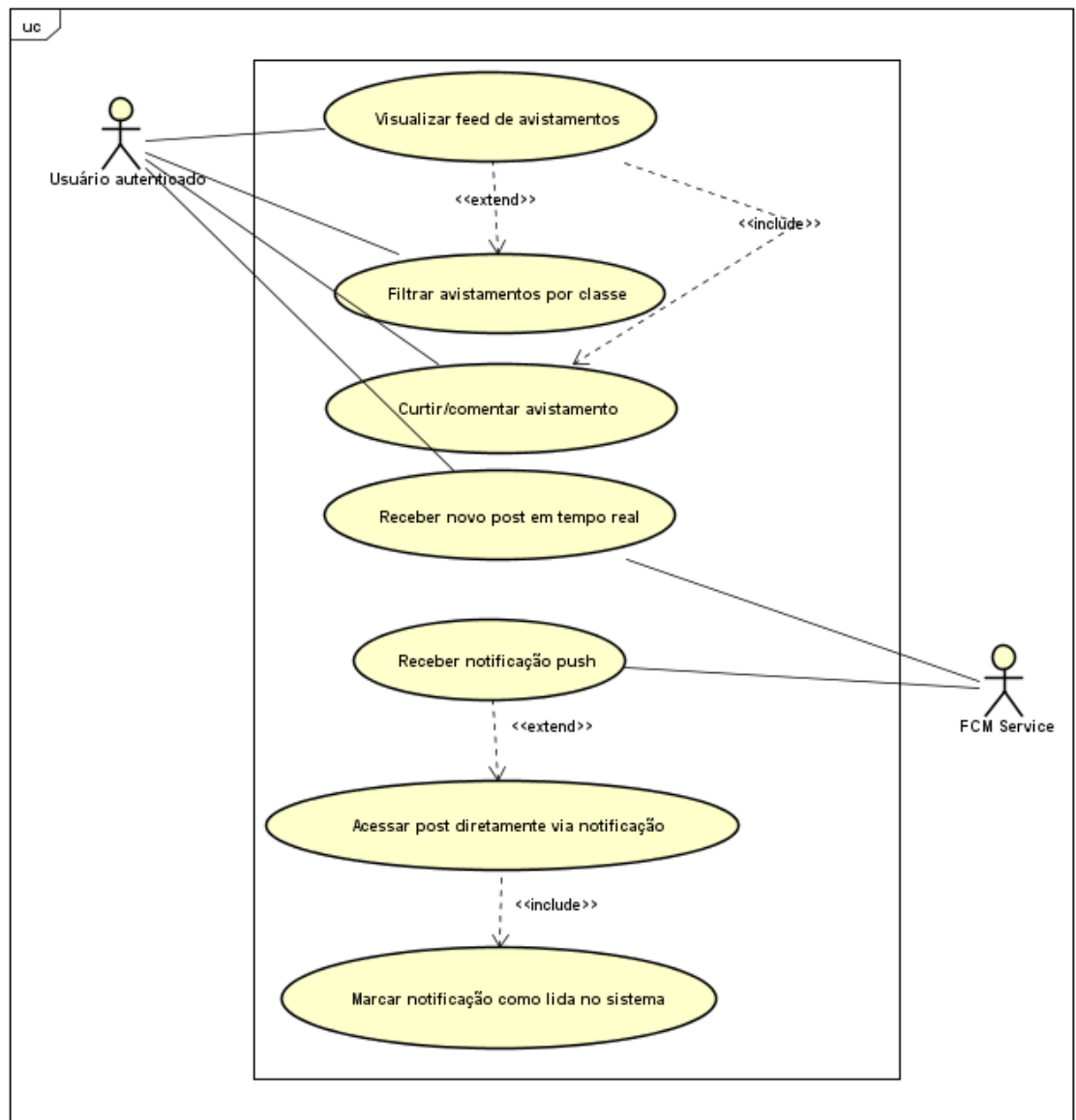
6.1 Casos de Uso

Caso de Uso Cadastrar Usuário;



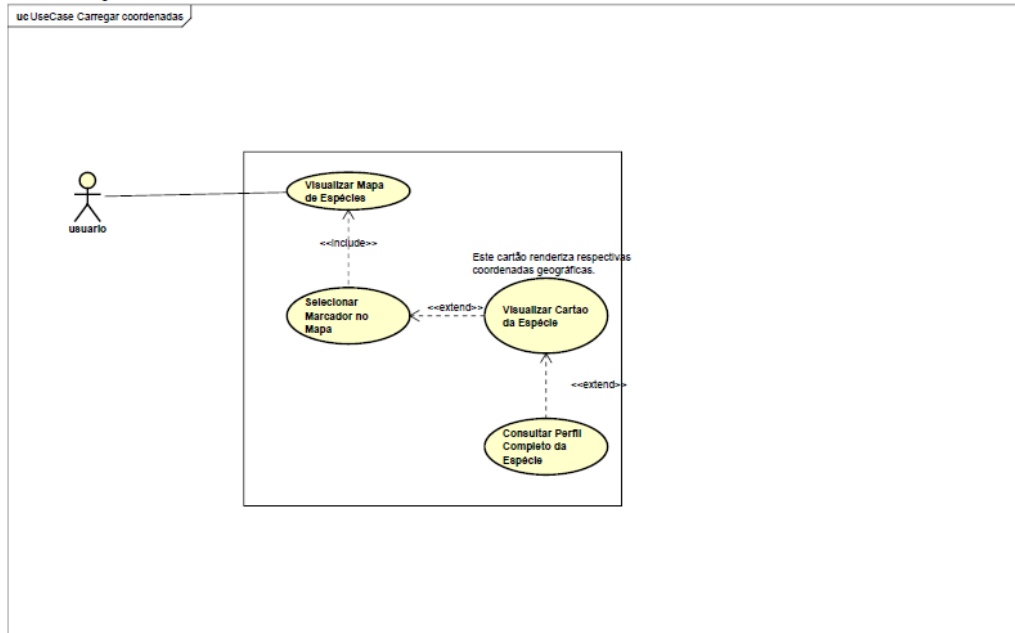
powered by Astah

Caso de Uso Visualizar Feed



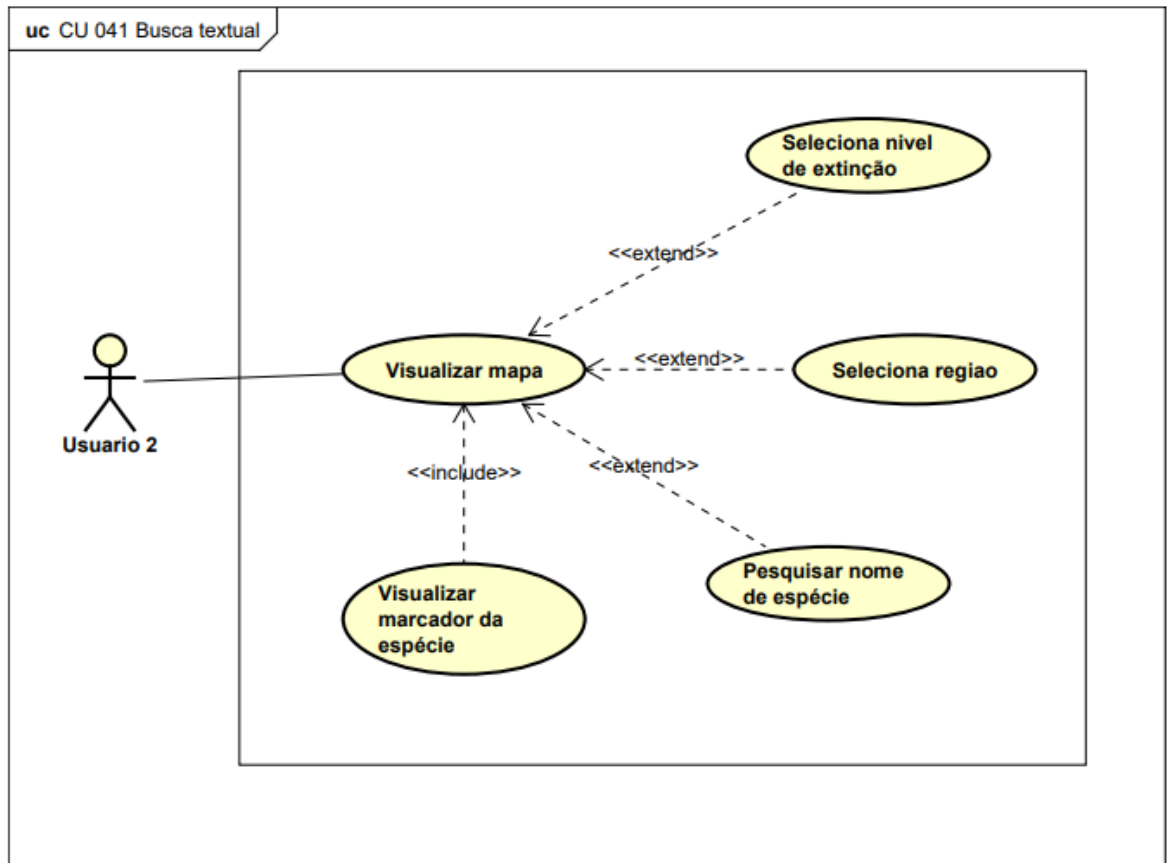
UseCase Carregar coordenadas

2026/06/10



CU 041 Busca textual

2026/06/11



6.1.2 Diagrama de Sequência

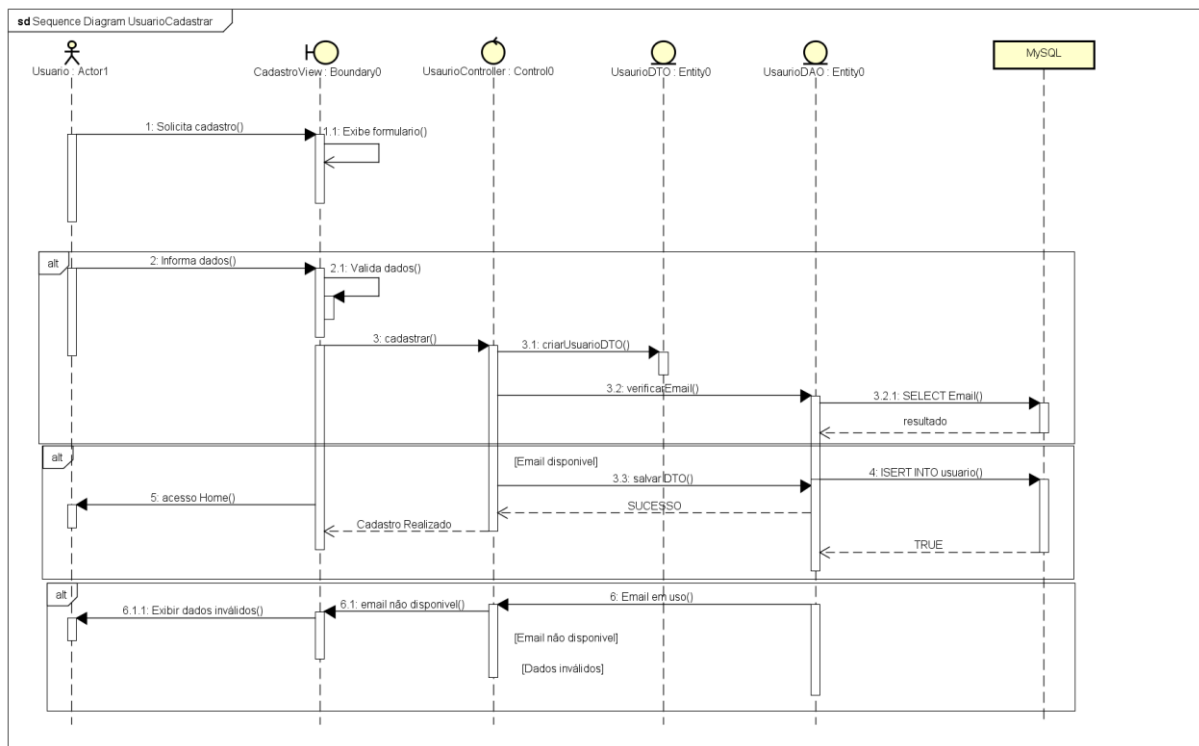
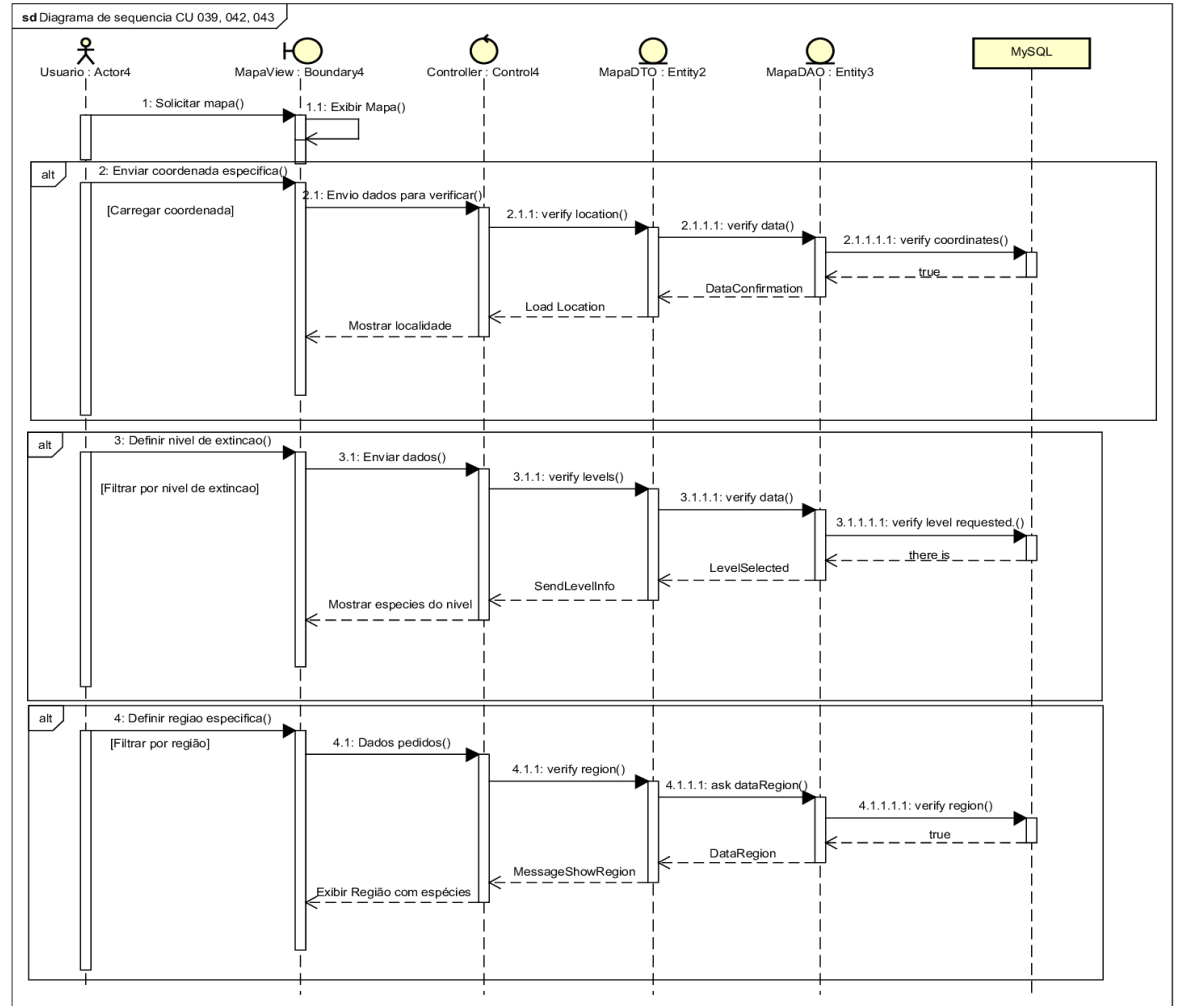


Figura 20 –Diagrama de Sequência Cadastro de Usuário

Diagrama de sequencia CU 039, 042, 043

2026/06/22



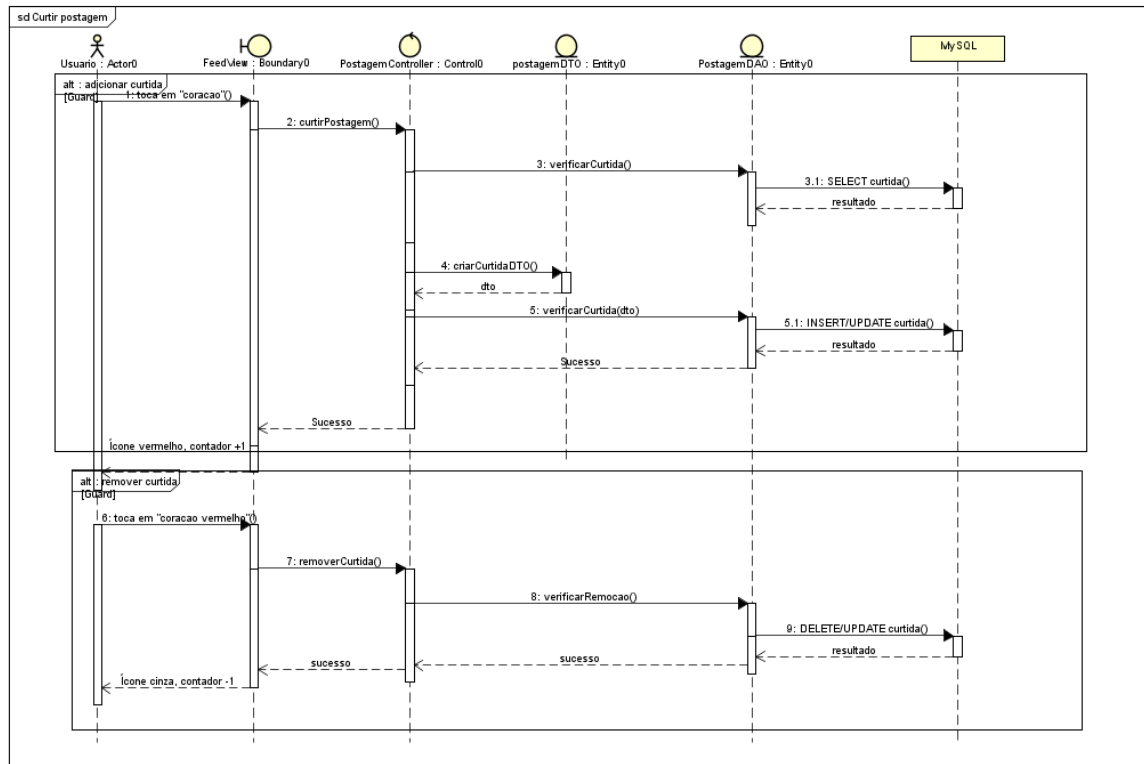


Diagrama de sequencia 046 048

2026/06/22

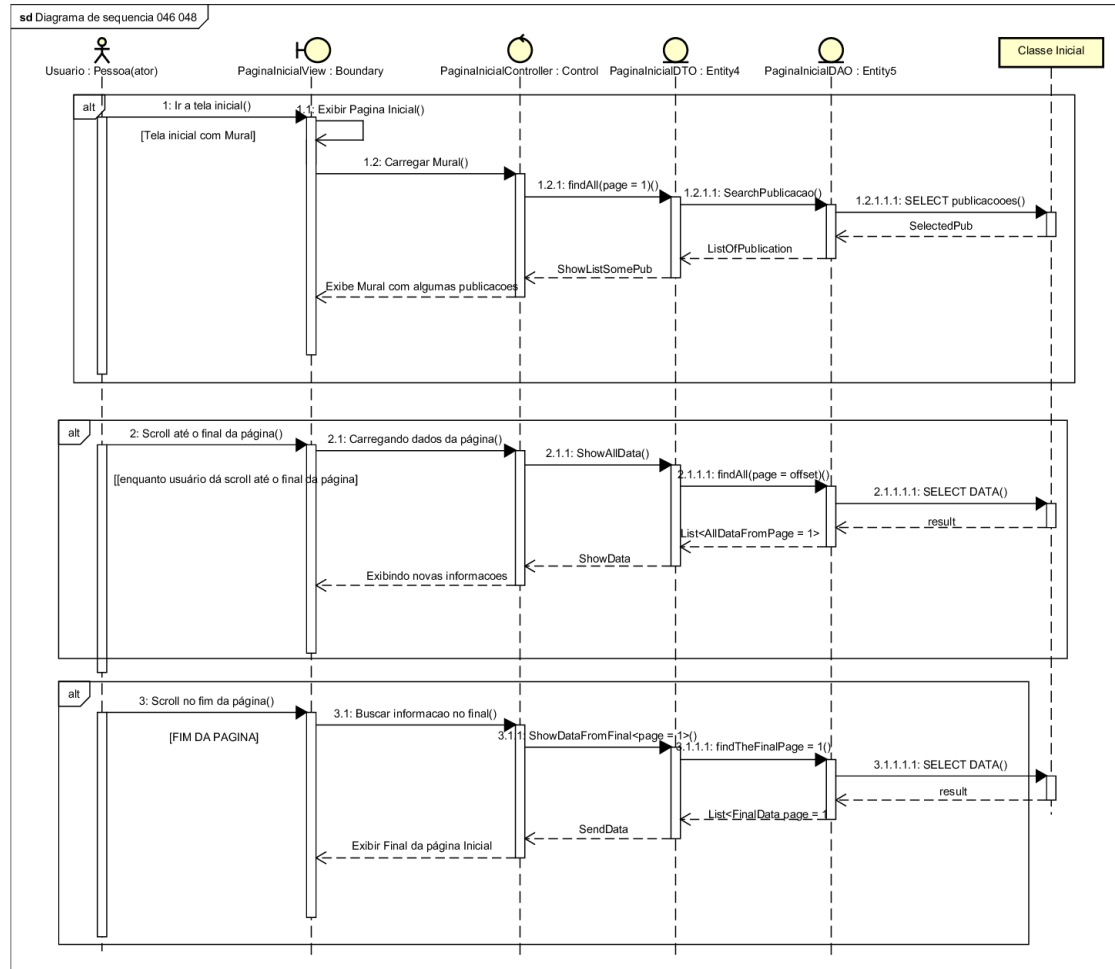
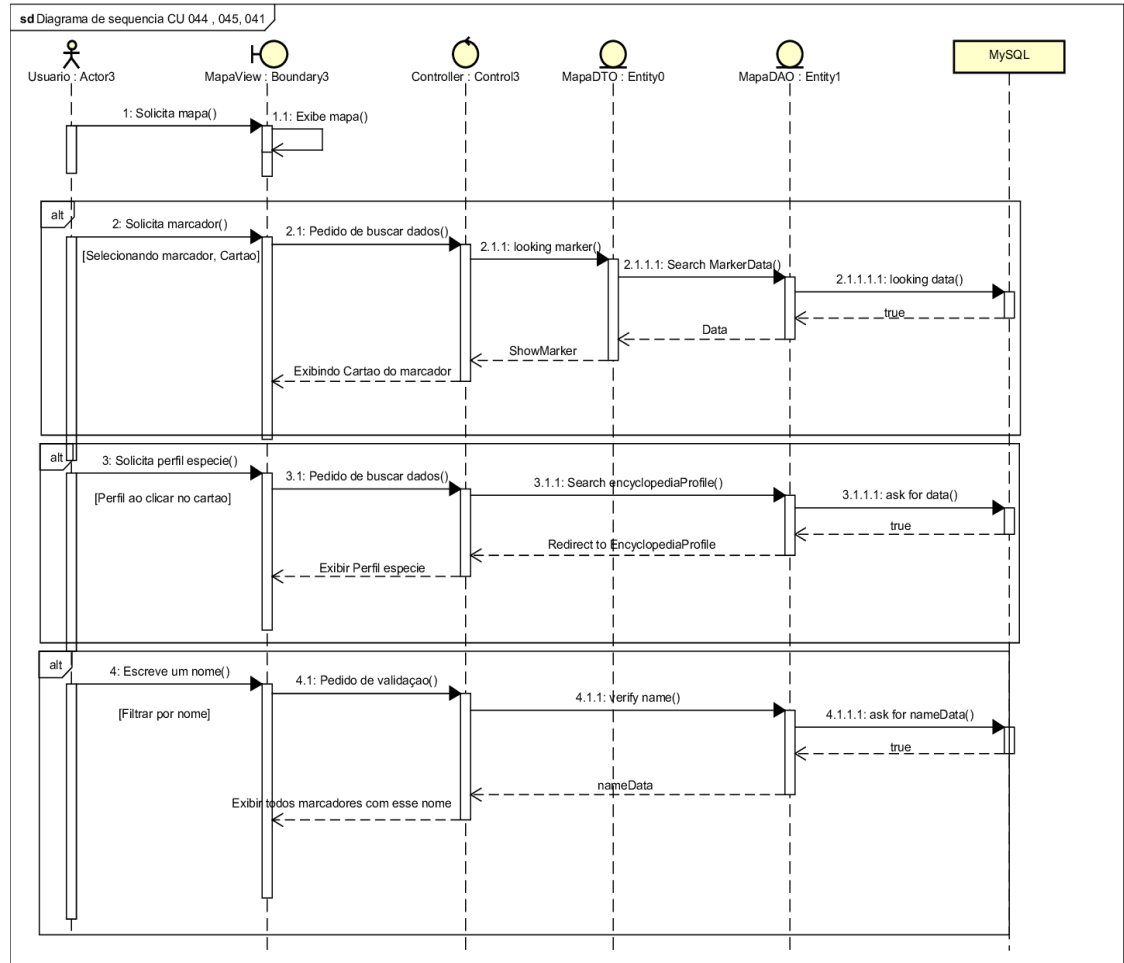
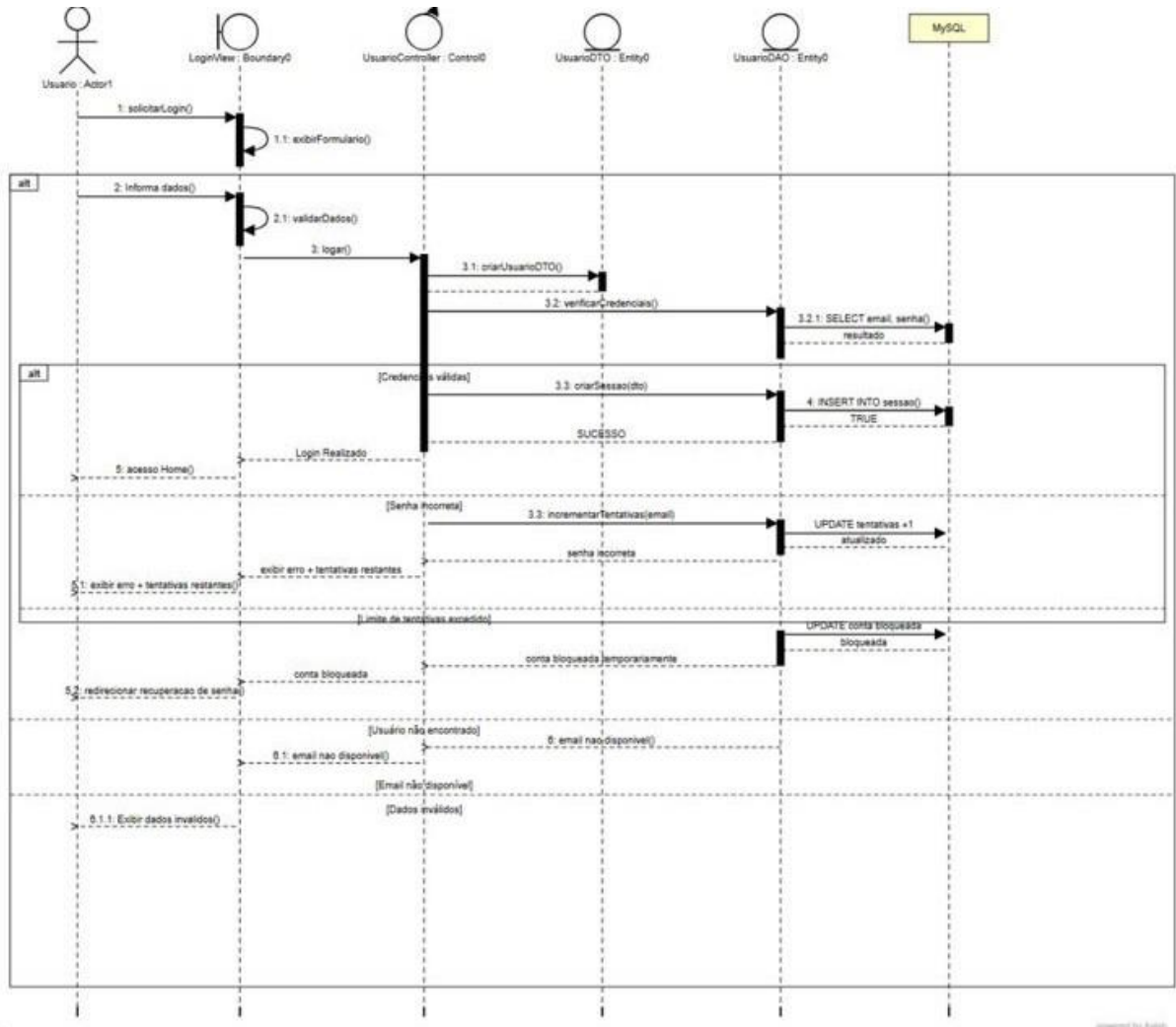
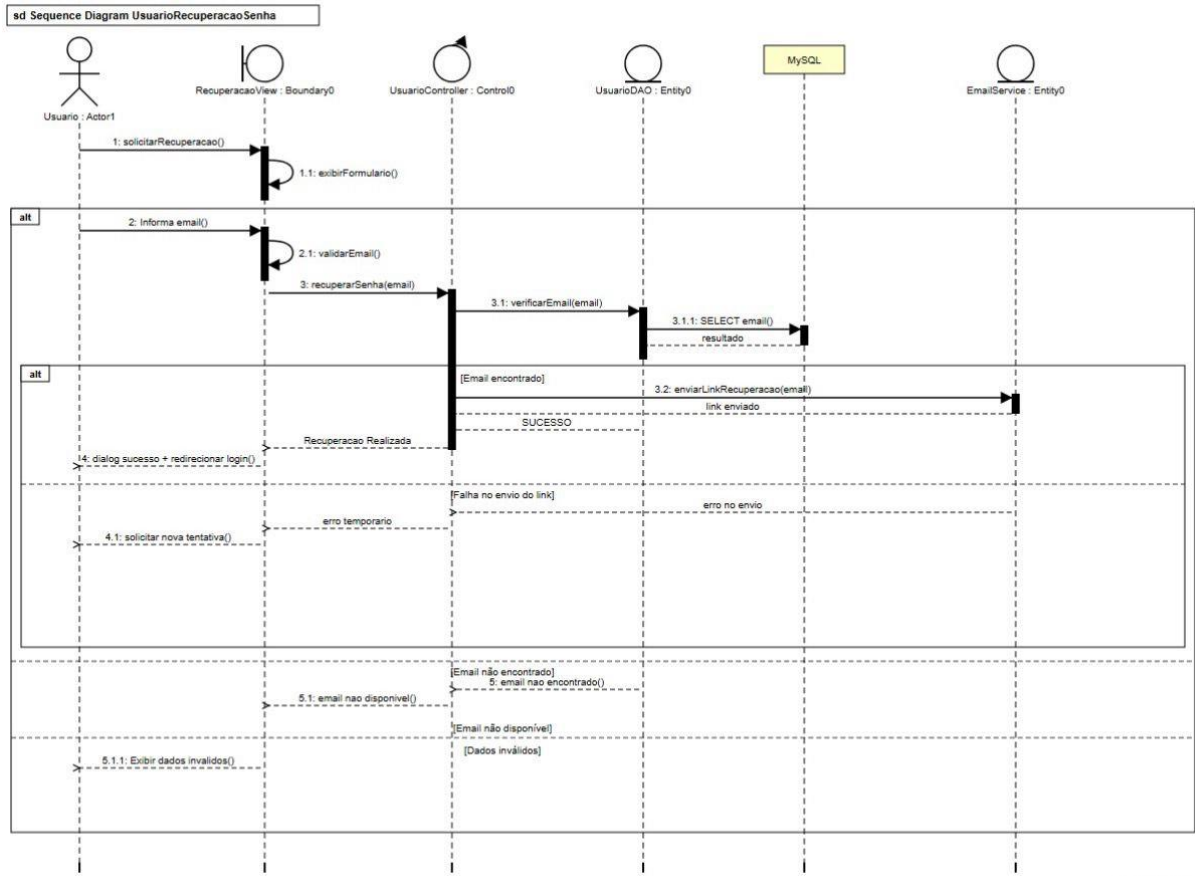


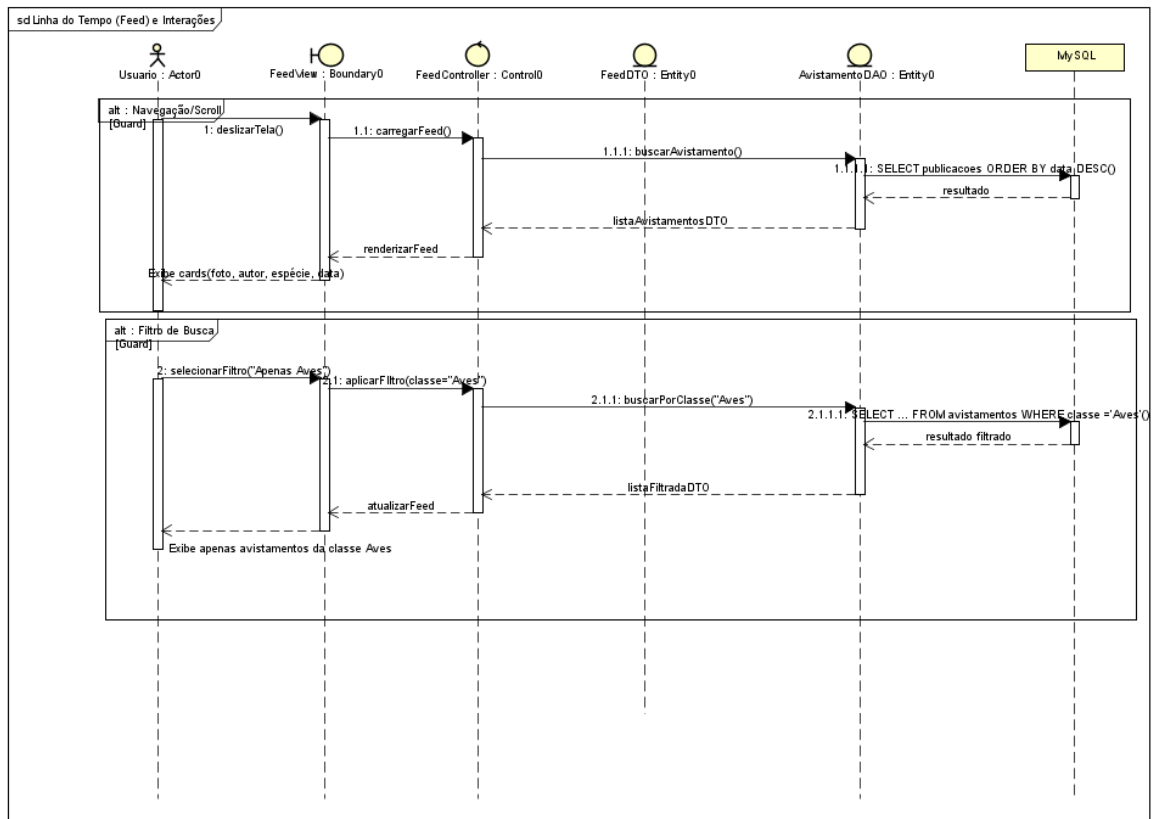
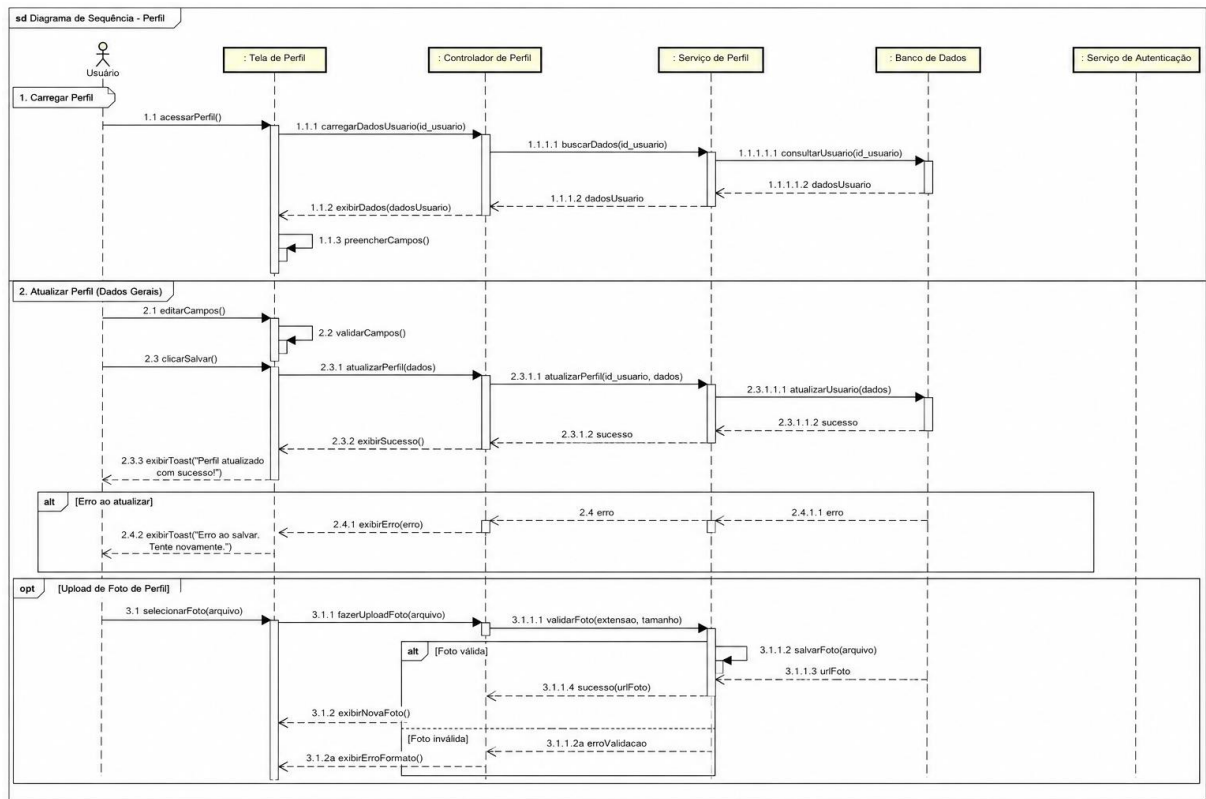
Diagrama de sequencia CU 044 , 045, 041

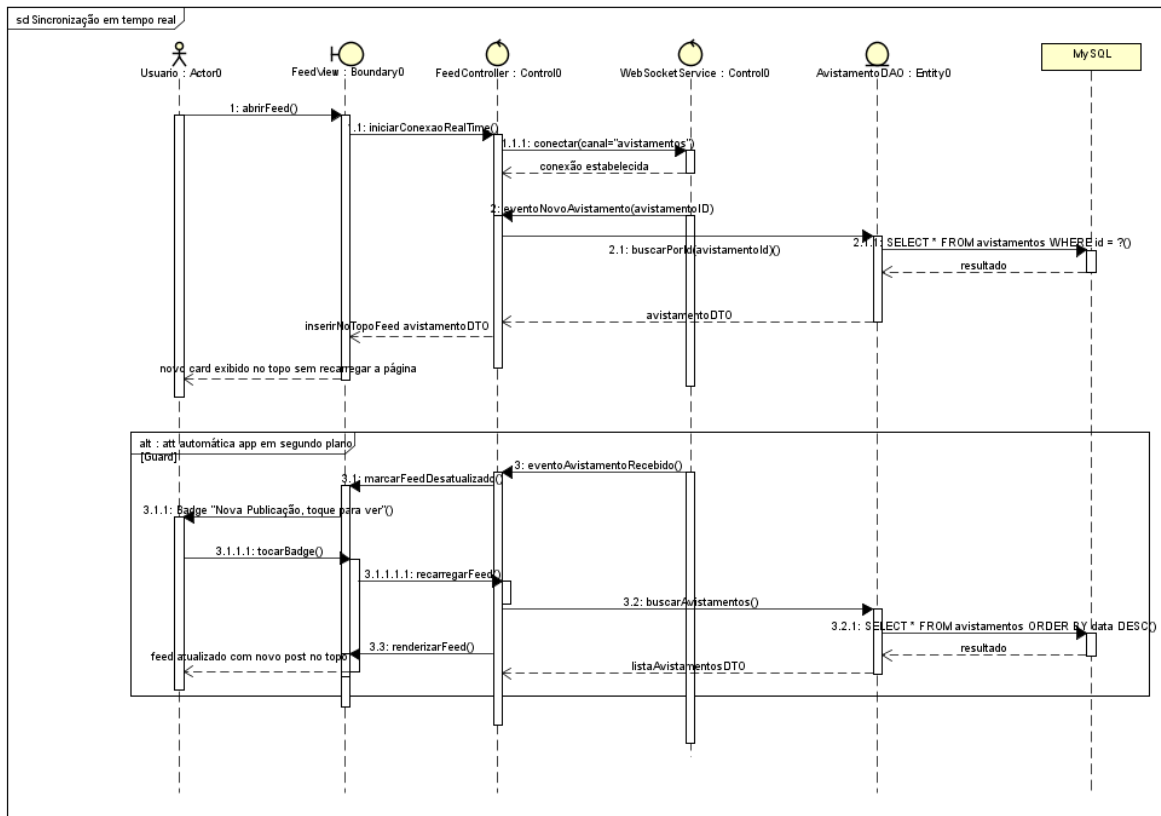
2026/06/22











7. VISÃO DE IMPLANTAÇÃO

2 Descrever os nodos físicos, as configurações e os artefatos que serão implantados.

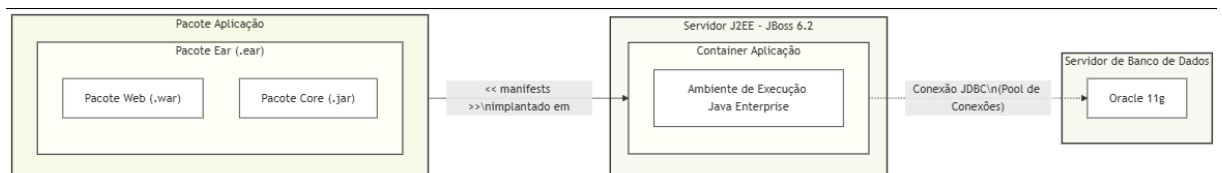


Figura 20 – Diagrama de Implantação Java

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Enumerar os itens relativos ao volume de acesso aos recursos da aplicação:

- a) Número de estimado usuários: 15.000
Justificativa: Projeção para o primeiro ano de operação da plataforma, considerando a adesão de cidadãos cientistas, pesquisadores, estudantes e entusiastas da conservação ambiental.
- b) Número estimado de acessos diários: 1.500 usuários ativos diários (DAU).
Justificativa: Baseia-se em uma taxa de engajamento padrão de 10% do total de usuários cadastrados acessando o sistema diariamente para interagir com o feed, consultar o mapa ou publicar novos avistamentos.
- c) Número estimado de acessos por período: 5.000 conexões simultâneas no período de pico.
Justificativa: Os períodos de maior intensidade ocorrem nos horários diurnos e de fins de semana (entre 10:00h e 17:00h), momento em que os usuários realizam atividades de campo e expedições, gerando requisições síncronas de processamento de imagens pela IA e plotagem de coordenadas no mapa. O teto de pico simultâneo é estimado em 20% dos acessos diários em uma mesma hora.
- d) Tempo de sessão de um usuário: 15 minutos por sessão.
Justificativa: Tempo médio estimado para que um usuário realize o fluxo completo na aplicação: abrir o app, autenticar-se (se necessário), carregar o feed, enviar uma nova postagem com foto para identificação taxonômica automatizada e visualizar o mapa de distribuição das espécies. Este valor serve para configurar o timeout de sessão no container de aplicação Spring Boot para expirar credenciais inativas e liberar memória.

8.2 Performance

Esta seção enumera as metas de desempenho e os tempos de resposta esperados para as transações do sistema, servindo como Critério de Aceitação para a homologação da arquitetura física (Spring Boot 3.4.1 e MySQL):

- 1. Tempo máximo para a execução de determinada transação:
 - a) Transações de Autenticação e Cadastro (Ex: Caso de Uso "Fazer Login" / "Criar Conta"): $\leq 1,5$ segundos.
 - b) *Detalhamento*: Tempo limite para a validação das credenciais pelo Spring Security, consulta de hash de segurança no MySQL e geração do token de sessão (JWT).
 - c) Transações de Consulta Comum (Ex: "Visualizar Feed"): $\leq 2,0$ segundos.
 - d) *Detalhamento*: Tempo para a camada REST buscar e paginar as postagens mais recentes e seus respectivos metadados biológicos na persistência.
 - e) Transações Geográficas (Ex: "Buscar Espécies no Mapa"): $\leq 2,5$ segundos.
 - f) *Detalhamento*: Tempo limite para a execução de queries espaciais no banco de dados e retorno dos marcadores de coordenadas (Latitude/Longitude) para renderização na Camada Cliente.
 - g) Transações de Processamento Pesado (Ex: Caso de Uso "Publicar Avistamento" com IA): $\leq 4,5$ segundos.
 - h) *Detalhamento*: Cenário mais crítico da aplicação. Este tempo engloba o upload do arquivo de imagem da foto, o processamento da inferência estatística de visão computacional na classe de serviço TFmodel, o vínculo taxonômico da Espécie e a gravação final do registro no MySQL.
- 2. Vazão Mínima do Sistema (Throughput):

- a) O servidor de aplicação Spring Boot deve ser capaz de processar no mínimo 80 requisições por segundo (RPS) sem apresentar degradação nos tempos de resposta acima estipulados.
- b) Tempo Máximo de Indisponibilidade Tolerada (SLA):
O sistema deve manter uma taxa de disponibilidade de 99,5% (Uptime). Qualquer travamento ou indisponibilidade do Container de Aplicação não pode exceder 15 minutos contínuos para recuperação automática dos serviços.
- c) Taxa Máxima de Erros Tolerada em Pico de Acesso:
Em cenários de concorrência máxima (até 5.000 conexões simultâneas no período de pico), a taxa de erros HTTP da camada de serviço REST (erros como 500 Internal Server Error ou 504 Gateway Timeout) deve ser inferior a 1% do total de requisições.

9. QUALIDADE

Enumerar os itens de qualidade de software [QOS] significativos para a aplicação:

Item	Descrição	Solução
Escalabilidade	Capacidade de absorver o aumento no volume de acessos (estimado em até 5.000 conexões simultâneas) e processar de forma eficiente o upload contínuo de imagens pesadas para análise taxonômica pela IA sem degradar a experiência do usuário.	Horizontalização e Assincronismo: Configuração de um cluster de balanceamento de carga (Load Balancer) distribuindo requisições entre múltiplas instâncias da aplicação Spring Boot. O processamento de imagens do módulo TFmodel será assíncrono via filas de mensageria (Spring AMQP/RabbitMQ), impedindo o travamento das threads síncronas dos controladores REST. No banco MySQL, serão aplicados índices compostos B-Tree e partições nas tabelas de coordenadas geográficas para consultas rápidas ao mapa.
Confiabilidade, Disponibilidade	Garantia de que o ecossistema permaneça operacional dentro da meta estabelecida de 99,5% de Uptime, permitindo que pesquisadores e usuários registrem avistamentos a qualquer momento sem falhas catastróficas ou perda de dados biológicos históricos.	Failover Automatizado e Resiliência: Configuração de múltiplas instâncias Spring Boot atrás de um Load Balancer, com sessões stateless via JWT (sem replicação de estado), permitindo que se uma instância cair, outra assuma transparentemente. Para o banco de dados, aplicação do <i>MySQL Group Replication</i> para replicação em tempo real em um servidor de contingência. Integrações externas (como envio de e-mails ou Mapas) usarão o padrão <i>Circuit Breaker</i> para isolar falhas de terceiros.
Portabilidade	Facilidade de implantar, migrar e executar os módulos do sistema em diferentes ambientes corporativos (como servidores de Desenvolvimento, Homologação e Produção) ou	Containerização com Spring Boot e Docker: Empacotamento da aplicação como um JAR executável único com servidor Tomcat embutido, garantindo que o artefato compilado rode de forma idêntica em qualquer ambiente compatível. Uso de imagens Docker para empacotar a aplicação Spring Boot junto às bibliotecas nativas necessárias para a execução do módulo TFmodel.

	fornecedores de nuvem sem necessidade de reescrever o código-fonte.	
Segurança	Proteção de dados sensíveis de cadastro dos usuários (senhas e e-mails), isolamento de privilégios de acordo com o tipo de conta (Cientista, Comum, Admin) e blindagem das rotas públicas da API de Serviço REST contra acessos maliciosos.	RBAC, Criptografia e Tokens: Autenticação stateless baseada em tokens JWT (JSON Web Tokens) assinados com chaves robustas (RS256). O controle de acesso a funcionalidades específicas será feito via RBAC (Role-Based Access Control) usando anotações Spring Security (ex: @PreAuthorize) vinculadas ao enum TipoDaConta. Armazenamento de senhas no MySQL utilizando algoritmo de hash seguro com sal (BCrypt) e imposição do protocolo TLS 1.3 (HTTPS) para tráfego de dados.